YapYap REST API Documentation

This document provides comprehensive documentation for the YapYap chat server REST API.

Base URL

All API endpoints are prefixed with `/api/v1`

Example: `http://localhost:8080/api/v1/auth/login`

Authentication

Most endpoints require JWT authentication. Include the JWT token in the Authorization header:

```
Authorization: Bearer <jwt_token>
```

JWT tokens are obtained through the `/auth/login` endpoint.

Admin User Configuration

Admin privileges are assigned by specifying user IDs in the `config.json` file. These users must already exist (created through normal registration) and will be assigned admin roles automatically at server startup:

```
{
  "admin_user_ids": [1, 2, 5]
}
```

Security Notes:

- Users must be created first through normal registration (`/api/auth/register`)
- Admin roles are assigned automatically at server startup
- Only existing admins can create new admin accounts via the API
- Admin roles grant all system management capabilities

Response Format

All responses are in JSON format. Error responses follow this structure:

```
{
  "error": "Error message description"
}
```

Endpoints

Authentication

POST /auth/login

Login with username and password to obtain a JWT token.

Request Body:

```
{
  "username": "string",
  "password": "string"
}
```

Response (200 OK):

```
{
  "user_id": 123,
  "token": "jwt_token_string",
  "user": {
    "id": 123,
    "username": "username",
    "status": 1,
    "last_active": "2025-06-30T12:00:00Z",
    "avatar_url": "https://example.com/avatar.jpg",
    "bio": "User bio",
    "created_at": "2025-01-01T00:00:00Z",
    "updated_at": "2025-06-30T12:00:00Z"
  }
}
```

POST /auth/register

Register a new user account.

Request Body:

```
{
  "username": "string",
  "password": "string",
  "bio": "string (optional)"
}
```

Response (201 Created):

```
{
  "user_id": 123,
  "token": "jwt_token_string",
  "user": {
    "id": 123,
    "username": "username",
    "status": 1,
    "last_active": "2025-06-30T12:00:00Z",
    "avatar_url": "",
    "bio": "User bio",
    "created_at": "2025-06-30T12:00:00Z",
    "updated_at": "2025-06-30T12:00:00Z"
  }
}
```

GET /users/me

Get current user information (requires authentication).

Response (200 OK):

```
{
  "id": 123,
  "username": "username",
  "status": 1,
  "last_active": "2025-06-30T12:00:00Z",
  "avatar_url": "https://example.com/avatar.jpg",
  "bio": "User bio",
  "created_at": "2025-01-01T00:00:00Z",
  "updated_at": "2025-06-30T12:00:00Z"
}
```

Admin Endpoints

Admin endpoints require authentication and admin privileges. Only users with admin permissions can access these endpoints.

POST /admin/users

Create a new admin user account.

Authentication Required: Yes (Admin only)

Request Body:

```
{
  "username": "string",
  "password": "string",
  "email": "string"
}
```

Response (201 Created):

```
{
  "message": "Admin user created successfully",
  "user_id": 123,
  "username": "newadmin"
}
```

Error Responses:

- `400` - Invalid JSON or missing required fields

- `401` - Authentication required
- `403` - Admin access required
- `409` - Username already exists

Channels

All channel endpoints require authentication.

GET /channels

List all channels the authenticated user has access to.

Response (200 OK):

```
[
  {
    "id": 1,
    "name": "general",
    "type": 0,
    "created_at": "2025-01-01T00:00:00Z",
    "messages": []
  },
  {
    "id": 2,
    "name": "random",
    "type": 0,
    "created_at": "2025-01-01T00:00:00Z",
    "messages": []
  }
]
```

Channel Types:

- `0` - Text Channel
- `1` - Direct Message
- `2` - Voice Channel
- `3` - Announcement Channel

POST /channels

Create a new channel (requires admin or manage channels permission).

Request Body:

```
{
  "name": "string",
  "type": 0
}
```

Response (201 Created):

```
{
  "id": 3,
  "name": "new-channel",
  "type": 0,
  "created_at": "2025-06-30T12:00:00Z",
  "messages": []
}
```

GET /channels/:id

Get a single channel by ID.

Response (200 OK):

```
{
  "id": 1,
  "name": "general",
  "type": 0,
  "created_at": "2025-01-01T00:00:00Z",
  "messages": []
}
```

PUT /channels/:id

Update a channel (requires admin or manage channels permission).

Request Body:

```
{
  "name": "updated-channel-name (optional)",
  "type": 0
}
```

Response (200 OK):

```
{
  "id": 1,
  "name": "updated-channel-name",
  "type": 0,
  "created_at": "2025-01-01T00:00:00Z",
  "messages": []
}
```

DELETE /channels/:id

Delete a channel (requires admin or manage channels permission).

Response (200 OK):

```
{
  "message": "Channel deleted"
}
```

GET /channels/:id/messages

Get messages from a channel. Results are ordered newest-first.

Query Parameters:

- `limit` (optional) - Number of messages to retrieve (default: 50, max: 100)
- `before` (optional) - Return messages older than the message with this ID
- `after` (optional) - Return messages newer than the message with this ID (returned newest-first)
- `around` (optional) - Return a window around the message with this ID (half newer + half older + pivot)

Use only one of `before`, `after`, or `around`.

Response (200 OK):

```
[
  {
    "id": 1,
    "channel_id": 1,
    "user_id": 123,
    "content": "Hello, world!",
    "attachments": ["https://example.com/file.jpg"],
    "created_at": "2025-06-30T12:00:00Z",
    "updated_at": "2025-06-30T12:00:00Z"
  }
]
```

PUT /channels/:id/permissions

Update channel permissions for a user (requires admin or manage channels permission).

Request Body:

```
{
  "user_id": 123,
  "view_channel": true,
  "send_message": true,
  "send_attachment": true,
  "manage_messages": false,
  "manage_channel": false
}
```

Response (200 OK):

```
{
  "id": 1,
  "channel_id": 1,
  "user_id": 123,
  "view_channel": true,
  "send_message": true,
  "send_attachment": true,
  "manage_messages": false,
  "manage_channel": false
}
```

Overwrites: Channel permission overwrites (Admin or Manage Channels)

These endpoints manage Discord-like per-channel allow/deny overwrites for roles or members.

Targets:

- `target_type` : `0` for role, `1` for member
- `target_id` : the role ID or user ID

GET /channels/:id/overwrites

List all overwrites for a channel.

Response (200 OK):

```
[
  {
    "id": 10,
    "channel_id": 1,
    "target_type": 0,
    "target_id": 3,
    "allow": 3,
    "deny": 0,
    "created_at": "2025-06-30T12:00:00Z"
  }
]
```

PUT /channels/:id/overwrites

Create or update a channel overwrite.

Request Body:

```
{
  "target_type": 0,
  "target_id": 3,
  "allow": 3,
  "deny": 0
}
```

`allow` and `deny` are bitmasks composed of permission flags (see Standards). For example, `3` is VIEW_CHANNEL | SEND_MESSAGES.

Response (201 Created | 200 OK):

Returns the created/updated overwrite.

DELETE /channels/:id/overwrites

Delete a channel overwrite.

Request Body:

```
{
  "target_type": 0,
  "target_id": 3
}
```

Response (200 OK):

```
{ "message": "Overwrite deleted" }
```

Messages

All message endpoints require authentication.

POST /messages

Create a new message.

Request Body:

```
{
  "channel_id": 1,
  "content": "Hello, world!",
  "attachments": ["https://example.com/file.jpg"] // optional
}
```

Response (201 Created):

```
{
  "id": 1,
  "channel_id": 1,
  "user_id": 123,
  "content": "Hello, world!",
  "attachments": ["https://example.com/file.jpg"],
  "created_at": "2025-06-30T12:00:00Z",
  "updated_at": "2025-06-30T12:00:00Z"
}
```

PUT /messages/:id

Update a message (requires message ownership or manage messages permission).

Request Body:

```
{
  "content": "Updated message content",
  "attachments": ["https://example.com/updated-file.jpg"]
}
```

Response (200 OK):

```
{
  "id": 1,
  "channel_id": 1,
  "user_id": 123,
  "content": "Updated message content",
  "attachments": ["https://example.com/updated-file.jpg"],
  "created_at": "2025-06-30T12:00:00Z",
  "updated_at": "2025-06-30T12:01:00Z"
}
```

DELETE /messages/:id

Delete a message (requires message ownership or manage messages permission).

Response (200 OK):

```
{
  "message": "Message deleted"
}
```

Users

GET /users

List all users (requires admin or manage users permission).

Authentication Required: Yes (Admin or manage users permission)

Query Parameters:

- `limit` (optional) - Number of users to retrieve (default: 50, max: 100)

Response (200 OK):

```
[
  {
    "id": 123,
    "username": "user1",
    "status": 1,
    "last_active": "2025-06-30T12:00:00Z",
    "avatar_url": "https://example.com/avatar1.jpg",
    "bio": "User 1 bio",
    "created_at": "2025-01-01T00:00:00Z",
    "updated_at": "2025-06-30T12:00:00Z"
  }
]
```

GET /users/:id

Get a user by ID.

Authentication Required: Yes

Response (200 OK):

```
{
  "id": 123,
  "username": "username",
  "status": 1,
  "last_active": "2025-06-30T12:00:00Z",
  "avatar_url": "https://example.com/avatar.jpg",
  "bio": "User bio",
  "created_at": "2025-01-01T00:00:00Z",
  "updated_at": "2025-06-30T12:00:00Z"
}
```

GET /users/:id/roles

Get roles assigned to a user.

Authentication Required: Yes

Response (200 OK):

```
[
  {
    "id": 1,
    "name": "Moderator",
    "permissions": {
      "manage_messages": "allow",
      "manage_channels": "deny",
      "admin": "unset"
    },
    "created_at": "2025-01-01T00:00:00Z",
    "updated_at": "2025-06-30T12:00:00Z"
  }
]
```

Roles

All role endpoints require admin permissions unless otherwise specified.

GET /roles

List all roles.

Authentication Required: Yes (Admin only)

Response (200 OK):

```
[
  {
    "id": 1,
    "name": "Moderator",
    "permissions": {
      "manage_messages": "allow",
      "manage_channels": "deny",
      "admin": "unset"
    },
    "created_at": "2025-01-01T00:00:00Z",
    "updated_at": "2025-06-30T12:00:00Z"
  }
]
```

POST /roles

Create a new role.

Authentication Required: Yes (Admin only)

Request Body:

```
{
  "name": "New Role",
  "permissions": {
    "manage_messages": "allow",
    "manage_channels": "deny",
    "admin": "unset"
  }
}
```

Permission Values:

- `"allow"` - Grant permission
- `"deny"` - Explicitly deny permission
- `"unset"` - No explicit setting (inherit from other sources)

Response (201 Created):

```
{
  "id": 2,
  "name": "New Role",
  "permissions": {
    "manage_messages": "allow",
    "manage_channels": "deny",
    "admin": "unset"
  },
  "created_at": "2025-06-30T12:00:00Z",
  "updated_at": "2025-06-30T12:00:00Z"
}
```

GET /roles/:id

Get a role by ID.

Authentication Required: Yes (Admin only)

Response (200 OK):

```
{
  "id": 1,
  "name": "Moderator",
  "permissions": {
    "manage_messages": "allow",
    "manage_channels": "deny",
    "admin": "unset"
  },
  "created_at": "2025-01-01T00:00:00Z",
  "updated_at": "2025-06-30T12:00:00Z"
}
```

PUT /roles/:id

Update a role.

Authentication Required: Yes (Admin only)

Request Body:

```
{
  "name": "Updated Role Name",
  "permissions": {
    "manage_messages": "allow",
    "manage_channels": "allow",
    "admin": "unset"
  }
}
```

Response (200 OK):

```
{
  "id": 1,
  "name": "Updated Role Name",
  "permissions": {
    "manage_messages": "allow",
    "manage_channels": "allow",
    "admin": "unset"
  },
  "created_at": "2025-01-01T00:00:00Z",
  "updated_at": "2025-06-30T12:01:00Z"
}
```

DELETE /roles/:id

Delete a role.

Authentication Required: Yes (Admin only)

Response (200 OK):

```
{
  "message": "Role deleted"
}
```

GET /roles/:id/users

Get all users assigned to a role.

Authentication Required: Yes (Admin only)

Response (200 OK):

```
[
  {
    "id": 123,
    "username": "user1",
    "status": 1,
    "last_active": "2025-06-30T12:00:00Z",
    "avatar_url": "https://example.com/avatar1.jpg",
    "bio": "User 1 bio",
    "created_at": "2025-01-01T00:00:00Z",
    "updated_at": "2025-06-30T12:00:00Z"
  }
]
```

POST /roles/assign

Assign a role to a user.

Authentication Required: Yes (Admin only)

Request Body:

```
{
  "user_id": 123,
  "role_id": 1
}
```

Response (201 Created):

```
{
  "user_id": 123,
  "role_id": 1
}
```

POST /roles/remove

Remove a role from a user.

Authentication Required: Yes (Admin only)

Request Body:

```
{
  "user_id": 123,
  "role_id": 1
}
```

Response (200 OK):

```
{
  "message": "Role removed from user"
}
```

Permissions

PUT /permissions/channels/:id

Update channel permissions for a user (same as PUT /channels/:id/permissions).

Authentication Required: Yes (Admin or manage channels permission)

Request Body:

```
{
  "user_id": 123,
  "view_channel": true,
  "send_message": true,
  "send_attachment": true,
  "manage_messages": false,
  "manage_channel": false
}
```

Response (200 OK):

```
{
  "id": 1,
  "channel_id": 1,
  "user_id": 123,
  "view_channel": true,
  "send_message": true,
  "send_attachment": true,
  "manage_messages": false,
  "manage_channel": false
}
```

Logs

All log endpoints require admin permissions.

GET /logs

Retrieve system logs with filtering options.

Query Parameters:

- `level` (optional) - Filter by log level (0=DEBUG, 1=INFO, 2=WARN, 3=ERROR, 4=FATAL)
- `action` (optional) - Filter by action type (e.g., "message.send", "user.login")
- `user_id` (optional) - Filter by user ID
- `target_id` (optional) - Filter by target resource ID
- `target_type` (optional) - Filter by target resource type
- `ip_address` (optional) - Filter by IP address
- `start_date` (optional) - Filter by date range (ISO 8601 format)
- `end_date` (optional) - Filter by date range (ISO 8601 format)
- `limit` (optional) - Number of logs to retrieve (default: 100, max: 1000)
- `offset` (optional) - Pagination offset

Response (200 OK):

```
{
  "logs": [
    {
      "id": 1,
      "level": 1,
      "action": "user.login",
      "message": "User logged in successfully",
      "user_id": 123,
      "target_id": null,
      "target_type": "",
      "ip_address": "192.168.1.100",
      "user_agent": "Mozilla/5.0...",
      "metadata": {
        "username": "testuser"
      },
      "created_at": "2025-06-30T12:00:00Z",
      "user": {
        "id": 123,
        "username": "testuser",
        "status": 1,
        "last_active": "2025-06-30T12:00:00Z",
        "avatar_url": "",
        "bio": ""
      }
    }
  ],
  "total_count": 1500,
  "limit": 100,
  "offset": 0
}
```


Log Levels:

- `0` - DEBUG
- `1` - INFO
- `2` - WARN
- `3` - ERROR
- `4` - FATAL

Common Action Types:

- `user.register` , `user.login` , `user.logout` , `user.update` , `user.delete`

- `channel.create` , `channel.update` , `channel.delete`
- `message.send` , `message.update` , `message.delete`
- `role.create` , `role.update` , `role.delete` , `role.assign` , `role.revoke`
- `permission.grant` , `permission.revoke`
- `system.startup` , `system.shutdown` , `system.error`
- `auth.success` , `auth.failure` , `auth.blocked`
- `websocket.connect` , `websocket.disconnect`

GET /logs/stats

Get log statistics and summary information.

Response (200 OK):

```
{
  "total_logs": 1500,
  "logs_by_level": {
    "DEBUG": 100,
    "INFO": 1200,
    "WARN": 150,
    "ERROR": 45,
    "FATAL": 5
  },
  "logs_by_action": {
    "user.login": 300,
    "message.send": 800,
    "channel.create": 25,
    "system.startup": 10
  },
  "recent_actions": [
    {
      "id": 1500,
      "level": 1,
      "action": "message.send",
      "message": "Message sent to channel 1",
      "user_id": 123,
      "created_at": "2025-06-30T12:00:00Z",
      "user": {
        "id": 123,
        "username": "testuser"
      }
    }
  ]
}
```


System

GET /health

Health check endpoint (no authentication required).

Response (200 OK):

```
{
  "status": "ok",
  "instance": "YapYap Development",
  "protocol_version": "0.1",
  "min_supported_version": "0.1"
}
```

GET /static/*

Serve static files from the assets directory (no authentication required).

Examples:

- `GET /static/index.html` - Serve index.html from assets/
- `GET /static/css/style.css` - Serve CSS files
- `GET /static/js/app.js` - Serve JavaScript files
- `GET /static/images/logo.png` - Serve image files

Response: Returns the requested static file or 404 if not found.

WebSocket Connection

In addition to the REST API, YapYap provides real-time communication via WebSocket.

WebSocket Endpoint: `/ws`

Authentication: Include JWT token as query parameter:

```
ws://localhost:8080/ws?token=your_jwt_token
```

The WebSocket connection is **server-to-client only** for real-time notifications. All client actions must use the REST API endpoints documented above.

Error Codes

- `400 Bad Request` - Invalid request format or missing required fields
- `401 Unauthorized` - Missing or invalid authentication token
- `403 Forbidden` - User lacks required permissions
- `404 Not Found` - Resource not found

- `409 Conflict` - Resource already exists (e.g., username taken, role already assigned)
- `500 Internal Server Error` - Server-side error

Rate Limiting

Currently, no rate limiting is implemented, but it's recommended for production deployments.

CORS

The WebSocket upgrader allows all origins by default. Configure CORS appropriately for your deployment environment.

Examples

Complete Authentication Flow

```
# Register a new user
curl -X POST http://localhost:8080/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username": "testuser", "password": "password123"}'

# Login (get token)
TOKEN=$(curl -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username": "testuser", "password": "password123"}' \
  | jq -r '.token')

# Use token for authenticated requests
curl -X GET http://localhost:8080/api/v1/users/me \
  -H "Authorization: Bearer $TOKEN"
```

Creating and Using Channels

```
# Create a channel
curl -X POST http://localhost:8080/api/v1/channels \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name": "general", "type": 0}'

# Send a message
curl -X POST http://localhost:8080/api/v1/messages \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"channel_id": 1, "content": "Hello, world!"}'

# Get channel messages
curl -X GET "http://localhost:8080/api/v1/channels/1/messages?limit=50" \
  -H "Authorization: Bearer $TOKEN"
```

Viewing Logs (Admin)

```
# Get recent logs
curl -X GET "http://localhost:8080/api/v1/logs?limit=10" \
  -H "Authorization: Bearer $ADMIN_TOKEN"

# Get error logs from today
curl -X GET "http://localhost:8080/api/v1/logs?level=3&start_date=2025-06-30T00:00:00Z" \
  -H "Authorization: Bearer $ADMIN_TOKEN"

# Get log statistics
curl -X GET http://localhost:8080/api/v1/logs/stats \
  -H "Authorization: Bearer $ADMIN_TOKEN"
```